

Mathematical Modeling of String Vibration Under Impact

Dylan He 2023141520101

Abstract

This report employees string vibration Partial Differential Equation to model the forced vibration of the tennis racket string. Two-dimensional Gaussian Function is applied to model the impact of the tennis. To model the damper, additional elastic forcing term and frictional forcing term are considered. The coupled dynamic system of the string and the damper is analyzed both analytically and numerically. Eigenfuntion expansion and Finite Difference Time Domain method are applied to get the behavior of the dynamic system. Finally, various parametric study and sensitivity analysis are conducted to further explain the stability of the dynamic system and the role of each parameter, also gave suggestions for optimizing the design of the damper.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.3	Objectives	4
1.4	Structure of the Report	4
2	Mathematical Modeling of String Vibration with Constrained Ends	5
2.1	Physical Assumptions	5
2.2	Equation of string vibration	5
2.3	Initial and Boundary Conditions	6

2.4	External Impact	6
2.5	Analytic Solution	7
2.6	Numerical Simulation	9
3	Modeling the Response of a String with a Damper	10
3.1	Damper	10
3.2	Modified Governing Equations	11
3.3	Boundary and Initial Conditions	11
3.4	Stability Analysis	12
3.5	Numerical Simulation	13
4	Discussion	14
4.1	Parametric Study	14
4.2	Sensitivity Analysis	16
5	Conclusions	18
6	Acknowledgments	18
7	References	19
A	Analytic Solution to the Wave Equation	20
B	Numerical Solution to the Wave Equation	23
C	Numerical Solution to the Coupled Dynamic System	25
D	Sensitivity Analysis	29

1 Introduction

1.1 Background and Motivation

In the realm of tennis sports, both recreational and professional players often seek equipment enhancements that can improve performance and reduce physical strain. One such enhancement is the use of a string-bed vibration damper. A vibration damper is a small device, typically made of rubber or silicone, designed to be attached to the strings of a tennis racquet. Its primary function is to absorb the shock generated when the ball impacts the racket, thereby reducing the vibrations transmitted through the racquet to the player's hand and arm.

When a tennis ball strikes the racquet, the strings vibrate, producing a high-pitched “ping” sound. The introduction of a vibration damper alters this acoustic response, diminishing the sharp “ping” and replacing it with a more subdued, low-pitched thud. This not only changes the auditory experience for the player but is also intended to minimize the transmission of vibrations that could lead to discomfort or injury over prolonged play.

The concept of damping in sports equipment is not new. Vibration dampers were first introduced by Rene Lacoste in 1964, with the initial premise that they could mitigate the shock of ball impact, thereby preventing injuries to the wrist and elbow. However, subsequent studies, such as a 2004 investigation published in the *Journal of Sports Sciences*, have questioned the efficacy of these dampers in reducing racket frame vibrations received at the forearm. Despite mixed empirical evidence regarding injury prevention, many players continue to use dampers primarily to eliminate the distracting pinging noise, thereby enhancing their concentration and focus during play.

1.2 Problem Statement

While vibration dampers are widely utilized in tennis racquets for their perceived benefits, the precise mechanisms by which they influence string vibrations remain inadequately understood. This project aims to mathematically model the vibration characteristics of a tennis racket string system under the influence of a string-bed damper. By employing partial differential equations (PDEs), the study seeks to analyze how the introduction of a damper affects the vibration patterns, energy dissipation, and overall dynamic response of the string system.

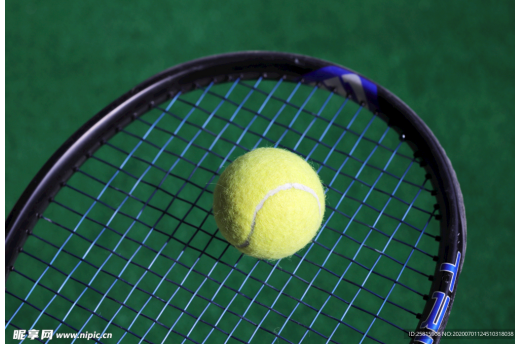


Figure 1: Tennis and racket

1.3 Objectives

The primary objectives of this study are as follows:

1. To develop a mathematical model representing the vibration of a tennis racket string with constrained ends.
2. To incorporate a damping mechanism into the model by simulating the effect of a string-bed damper.
3. To evaluate the impact of the damper on reducing vibrations and analyze optimal damping parameters for maximum effectiveness.

1.4 Structure of the Report

This report is organized into several sections, each addressing a specific aspect of the problem:

1. **Mathematical Modeling of String Vibration with Constrained Ends:** Establishes the foundational wave equation governing string vibrations and solves it under fixed boundary conditions.
2. **Modeling the Response of a String with a Damper:** Extends the basic model to include damping effects introduced by a vibration damper.
3. **Evaluation of Damping Effect and Optimization of Parameters:** Analyzes the effectiveness of damping in reducing vibrations and seeks optimal parameters for damper performance.

4. **Validation and Verification:** Compares analytical and numerical results to ensure the reliability of the model.
5. **Conclusions:** Summarizes the findings, discusses their implications, and suggests avenues for future research.

2 Mathematical Modeling of String Vibration with Constrained Ends

2.1 Physical Assumptions

- String is uniform, flexible, thin, and under tension.
- As a rule of thumb, the strings of tennis rackets are taut. Therefore, the displacement of the string is considered as a small displacement.
- Fixed ends.

2.2 Equation of string vibration

The classical wave equation is applied to model the vibration of string. Firstly, we set the coordinates of the string as shown in Figure 2. For a

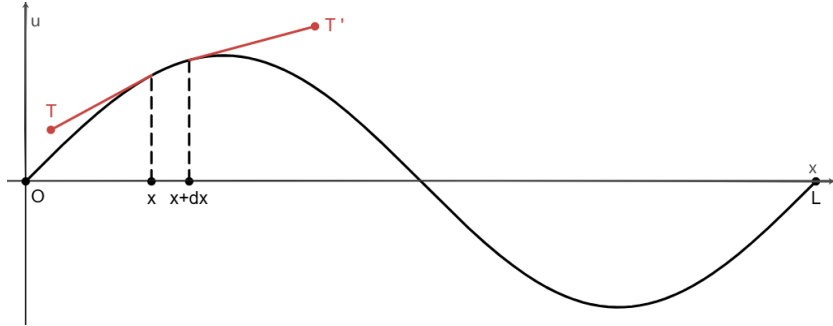


Figure 2: Coordinate of the string

small segment of string, the length of arc is given by

$$\int_x^{x+\Delta x} \sqrt{(dx)^2 + \left(\frac{\partial u}{\partial x} dx\right)^2} = \int_x^{x+\Delta x} \sqrt{1 + \left(\frac{\partial u}{\partial x}\right)^2} dx = \Delta x$$

where $\frac{\partial u}{\partial x} = 0$ because of the small displacement. This indicates that the string does not elongate during the small vibrations on the small segment. Besides, the segment has only vertical movement. Therefore, the tension on ends of the segment are equal $T = T'$. For the point at x , the net force is given by

$$\begin{aligned} T \sin \alpha(x + \Delta x, t) - T \sin \alpha(x, t) &\approx T(\tan \alpha(x + \Delta x, t) - \tan \alpha(x, t)) \\ &= T\left(\frac{\partial u}{\partial x}\Big|_{x+\Delta x} - \frac{\partial u}{\partial x}\Big|_x\right) \\ &= T\Delta x \frac{\partial^2 u}{\partial x^2} \end{aligned}$$

According to the Newton's second law:

$$\rho \Delta x \frac{\partial^2 u}{\partial t^2} = T \Delta x \frac{\partial^2 u}{\partial x^2}$$

where ρ is the linear density of the string. Assume the density of external force to be $F(x, t)$, and $f(x, t) = F(x, t)/\rho$. Then the governing equation of the vibration is given by

$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2} + f(x, t)$$

where $a^2 = \frac{T}{\rho}$

2.3 Initial and Boundary Conditions

- Boundary Condition:

$$u(0, t) = 0, \quad u(L, t) = 0 \quad \text{for all } t \geq 0$$

- Initial Condition:

$$u(x, 0) = 0, \quad \frac{\partial u}{\partial t}(x, 0) = 0$$

2.4 External Impact

According to the touch process of tennis, the spatiotemporal distribution of external force can be given by **two-dimensional Gaussian Distribution**.

$$f(x, t) = F_0 \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_x^2} + \frac{(t - t_0)^2}{2\sigma_t^2}\right)\right)$$

According to the references, we take the radius of tennis $r = 6.86 \text{ cm}$, the duration of touch $t_d = 0.004 \text{ s}$, the mass of tennis $m = 57 \text{ g}$, and the impact velocity $v_d = 45 \text{ m s}^{-1}$. Thus the parameter can be determined: $\sigma_x = \sqrt{2}r/3$, $\sigma_t = t_d/3$. Assume $N = 20$ strings are touched by the tennis the velocity is reversed after touching, then the total impulse is:

$$I = \rho N F_0 \int_{-\infty}^{\infty} \exp\left(-\frac{(x-x_0)^2}{2\sigma_x^2}\right) dx \cdot \int_{-\infty}^{\infty} \exp\left(-\frac{(t-t_0)^2}{2\sigma_t^2}\right) dt = \rho F_0 \cdot 2\pi\sigma_x\sigma_t$$

Therefore the parameter F_0 is given by

$$F_0 = \frac{9mv_d}{\sqrt{2}N\rho\pi r t_d}$$

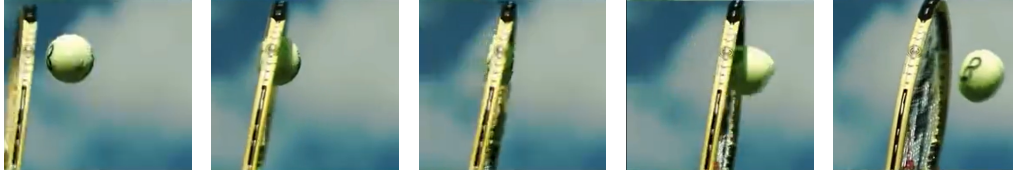


Figure 3: Tennis touch process

2.5 Analytic Solution

The eigenfunction expansion method is applied to solve the following deterministic problem

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2} + f(x, t) & (0 < x < L, t > 0) \\ u|_{x=0} = 0, \quad u|_{x=L} = 0 & (t > 0) \\ u|_{t=0} = 0, \quad \frac{\partial u}{\partial t}|_{t=0} = 0 \end{cases}$$

Firstly, consider the associated homogeneous equation

$$\frac{\partial^2 u}{\partial t^2} = \frac{T}{\rho} \frac{\partial^2 u}{\partial x^2}$$

Assume a solution of the form:

$$u(x, t) = X(x)T(t)$$

Substituting into the homogeneous equation and rearrange the equation:

$$\frac{T''(t)}{T(t)} = a^2 \frac{X''(x)}{X(x)} = -\lambda$$

Where $-\lambda$ is the separation constant. This leads to two ODE. We are particularly interested in the spatial one.

$$X''(x) + \lambda X(x) = 0$$

Considering the boundary condition, it's easy to find that the eigenfunction $X_n(x)$ is:

$$X_n(x) = \sin\left(\frac{n\pi x}{L}\right)$$

Then we consider the original PDE. Take the eigenfunction expansion on the solution:

$$u(x, t) = \sum_{n=1}^{\infty} g_n(t) \sin\left(\frac{n\pi x}{L}\right)$$

It's also an Half-range Fourier Series of $u(x, t)$ such that

$$g_n(t) = \frac{2}{L} \int_0^L u(x, t) \sin\left(\frac{n\pi x}{L}\right) dx$$

Back substitute into the PDE, we have:

$$\sum_{n=1}^{\infty} [g_n''(t) + \left(\frac{na\pi}{L}\right)^2 g_n(t)] \sin\left(\frac{n\pi x}{L}\right) = f(x, t)$$

Also expand $f(x, t)$ in terms of eigenfunctions:

$$f(x, t) = \sum_{n=1}^{\infty} f_n(t) \sin\left(\frac{n\pi x}{L}\right)$$

Then we have following equation

$$\sum_{n=1}^{\infty} [g_n''(t) + \left(\frac{na\pi}{L}\right)^2 g_n(t) - f_n(t)] \sin\left(\frac{n\pi x}{L}\right) = 0$$

For the equation to hold, the coefficient of the eigenfunction must be zero. Then this relation and initial condition compose an IVP of ODE.

$$\begin{cases} g_n''(t) + \left(\frac{na\pi}{L}\right)^2 g_n(t) - f_n(t) = 0 \\ g_n(0) = 0, \quad g_n'(0) = 0 \end{cases}$$

Take the Laplace transform

$$s^2 G + \left(\frac{na\pi}{L}\right)^2 G - F_n(s) = 0$$

$$G = \frac{F_n(s)}{s^2 + \left(\frac{na\pi}{L}\right)^2}$$

where $G = \mathcal{L}\{g_n(t)\}$ and $F_n(s) = \mathcal{L}\{f_n(t)\}$. Take inverse Laplace transform, we got the final solution to the PDE.

$$u(x, t) = \sum_{n=1}^{\infty} \left[\frac{L}{na\pi} \int_0^t f_n(\tau) \sin\left(\frac{na\pi}{L}(t - \tau)\right) d\tau \right] \sin\left(\frac{n\pi x}{L}\right)$$

$$f_n(\tau) = \frac{2}{L} \int_0^L F_0 \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_x^2} + \frac{(\tau - t_0)^2}{2\sigma_t^2}\right)\right) \sin\left(\frac{n\pi x}{L}\right) dx$$

2.6 Numerical Simulation

Finite Difference Time Domain (FDTD) is applied to simulate the vibration. We Discretize the Spatiotemporal domain into grids: $x_i = i\Delta x$, for $i = 0, 1, 2, \dots, N$ and $t^n = n\Delta t$, for $n = 0, 1, 2, \dots, M$.

Then approximate derivatives using central differences:

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{(\Delta t)^2}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

Rearrange the equation, we have the following updating rule:

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + \left(\frac{T}{\rho} \frac{(\Delta t)^2}{(\Delta x)^2}\right) (u_{i+1}^n - 2u_i^n + u_{i-1}^n) + (\Delta t)^2 f_i^n$$

The results of numerical simulation are shown in Figure 4 and Figure 5. Without damping the string keeps oscillating.

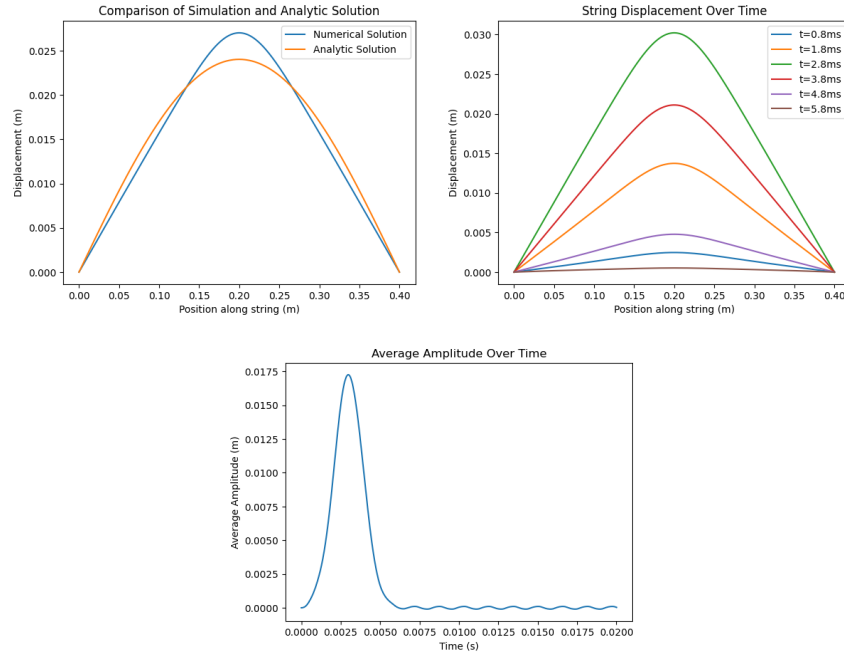


Figure 4: Results of Numerical Simulation and Comparison

3 Modeling the Response of a String with a Damper

3.1 Damper

As given in the Introduction, the damper is a small block that facilitates energy dissipation. It is believed to be able to significantly reduce the vibration. It is reasonable to assume that the damper impedes the displacing of string in the following two ways:

- **Elastic force:** Due to the relative displacement between the damper and the string.
- **Frictional force:** Due to the relative velocity between the damper and the string.

Let the amplitude of the damper be y . For a small segment of string, the elastic force is given by

$$F_e = E(y(t) - u(x, t))\Delta x$$

where E is the Young's Module of the damper. The frictional force is given by

$$F_f = \mu \left(\frac{\partial y}{\partial t} - \frac{\partial u}{\partial t} \right) \Delta x$$

where μ is the damping coefficient.

3.2 Modified Governing Equations

The governing equation of vibration changes due to the modification of the external forcing term.

$$\rho \Delta x \frac{\partial^2 u}{\partial t^2} = \Delta x \left(T \frac{\partial^2 u}{\partial x^2} + f(x, t) \rho \right) + W(x_d, d)(F_f + F_e)$$

where $W(x_d, d)$ is the window function

$$W(x_d, d) = \begin{cases} 1, & x \in (x_d, x_d + d) \\ 0, & \text{elsewhere} \end{cases}$$

x_d is the left end of the damper, and d is its width. According to the Newton's second law, the amplitude of the damper can be given by following equation:

$$m_d \frac{d^2 y}{dt^2} = - \sum W(x_d, d)(F_f + F_e)$$

where m_d is the mass of the damper. The final dynamic system is shown below

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} &= a^2 \frac{\partial^2 u}{\partial x^2} + f(x, t) + \frac{W(x_d, d)}{\rho} (E(y - u) + \mu \left(\frac{dy}{dt} - \frac{\partial u}{\partial t} \right)) \\ \frac{d^2 y}{dt^2} &= -\frac{1}{m_d} \int_0^L W(x_d, d) (E(y - u) + \mu \left(\frac{dy}{dt} - \frac{\partial u}{\partial t} \right)) dx \end{cases}$$

3.3 Boundary and Initial Conditions

- Boundary Condition:

$$u(0, t) = 0, \quad u(L, t) = 0 \quad \text{for all } t \geq 0$$

- Initial Condition:

$$u(x, 0) = 0, \quad \frac{\partial u}{\partial t}(x, 0) = 0$$

$$y(0) = 0, \quad \frac{dy}{dt} = 0$$

3.4 Stability Analysis

To theoretically indicate the effect of the damper that dissipates energy, we use the energy method to analyze the stability of the dynamic system. For a small segment of string, its kinetic energy is

$$dK = \frac{1}{2}\rho\Delta x \left(\frac{\partial u}{\partial t}\right)^2$$

From 0 to L , the whole kinetic energy:

$$K = \frac{1}{2} \int_0^L \rho \left(\frac{\partial u}{\partial t}\right)^2 \sqrt{1 + \left(\frac{\partial u}{\partial x}\right)^2} dx \approx \frac{1}{2} \int_0^L \rho \left(\frac{\partial u}{\partial t}\right)^2 dx$$

The potential energy of the string is given by

$$V = T\Delta L$$

$$\Delta L = \int_0^L \sqrt{1 + \left(\frac{\partial y}{\partial x}\right)^2} dx - L$$

Because the assumption of small displacement, which is validated by the previous numerical simulation, we apply the approximation $\sqrt{1 + \left(\frac{\partial y}{\partial x}\right)^2} \cong 1 + \frac{1}{2} \left(\frac{\partial y}{\partial x}\right)^2$. Therefore, the potential energy is given by

$$V(t) = \frac{1}{2} \int_0^L T \left(\frac{\partial y}{\partial x}\right)^2 dx$$

Considering the damper, the kinetic energy and elastic potential energy are

$$K_d = \frac{1}{2}m_d \left(\frac{dy}{dt}\right)^2$$

$$V_d = \frac{1}{2}E \int_{x_d}^{x_d+d} (y - u)^2 dx$$

The total energy for this dynamic system is given by

$$\mathcal{E} = \frac{1}{2} \int_0^L \left(\left(\frac{\partial u}{\partial t}\right)^2 + a^2 \left(\frac{\partial u}{\partial x}\right)^2 \right) dx + \frac{1}{2}m_d \left(\frac{dy}{dt}\right)^2 + \frac{1}{2}E \int_{x_d}^{x_d+d} (y - u)^2 dx$$

The damper introduces the dissipation of energy.

$$\frac{d\mathcal{E}}{dt} = \int_0^L \rho f(x, t) \frac{\partial u}{\partial t} dx - \frac{\mu}{\rho} \int_{x_d}^{x_d+d} \left(\frac{dy}{dt} - \frac{\partial u}{\partial t} \right)^2 dx$$

It's clear that the dissipative term is always negative when $\mu > 0$. Therefore, the damper can always dissipate the total energy of the dynamic system, thus reducing the vibration.

3.5 Numerical Simulation

The numerical simulation is conducted with FDTD method. In the simulation, the damper is set at the middle of the string, which means $x_d = \frac{L}{2}$. With the incorporation of the damper, the vibration of the string is less intensive and shape are significantly changed.

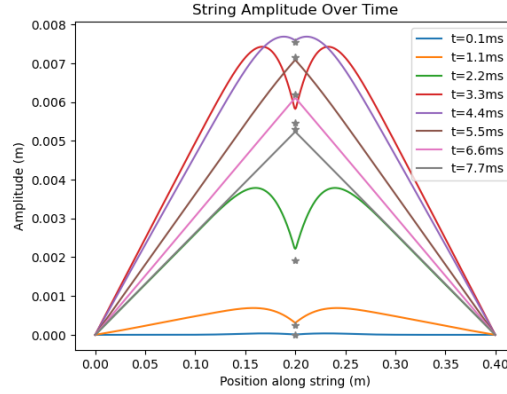


Figure 5: String Amplitude Over Time, damper is gray '*'

What's more, the phase trajectory diagram shows a trajectory spirals inward towards the equilibrium point. It shows that the initial and continuous perturbation leads to oscillatory response, and the damping effect reduces the amplitude. The combined effects of elastic coupling and damping ensure that oscillations decay, and the system asymptotically converges to the equilibrium point.

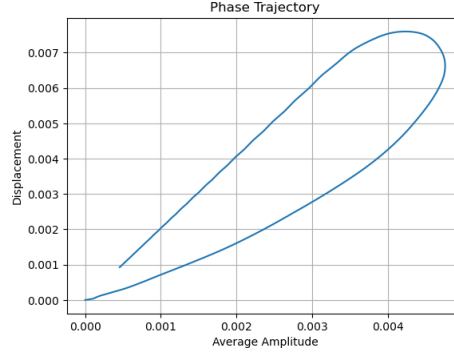


Figure 6: Phase Trajectory

Also the displacement of the damper and the amplitude of the string are coupled, as shown in Figure 7. This is due to the elastic force term.

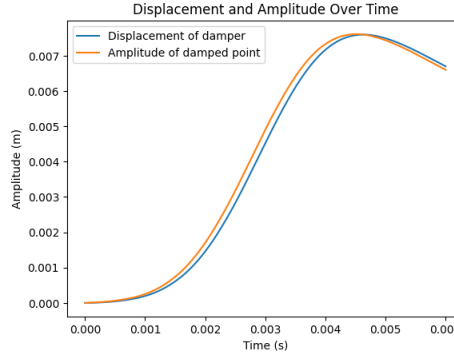


Figure 7: The Relative Displacement

In summary, the numerical simulation shows that damping effect is significant. With the same physical parameters, the max amplitude of vibration is much smaller with the damper, see Figure 8.

4 Discussion

4.1 Parametric Study

The most critical parameters that influence the behavior of the dynamic system are:

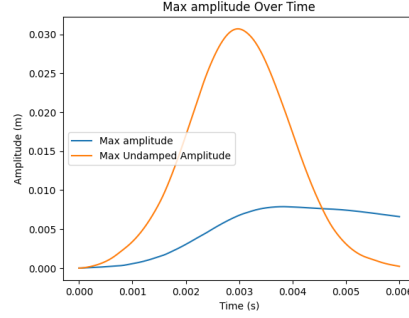


Figure 8: Comparison of Max Amplitude

- Damping Coefficient μ
- Elastic Module E
- Position of the damper x_d
- Width of the damper d

Multiple numerical simulations are conducted to study the role of parameters in determining the behavior of the dynamic system.

Elastic force and the frictional force is keys to the mode of vibration. Elastic module and the damping coefficient describes how strong the string coupled with the damper. Frictional forces impede the high frequency vibration. In Figure 9, with no elastic force and little damping effect, the damper also manages to vanish the vibration. When μ is less than around 20, tiny vibration can be observed (underdamped).

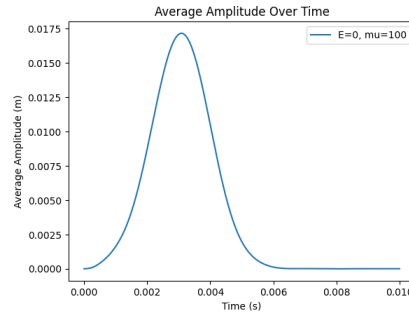


Figure 9: $\mu = 100 \text{ Pa s}$, $E = 0 \text{ Pa}$

Obviously, the position of the damper significantly influence the effectiveness of damping. Closer to the impact position, stronger the movement of string is, then more energy can be dissipated by the damper. When $x_d \rightarrow 0$ the dynamic system degraded as simple forced vibration.

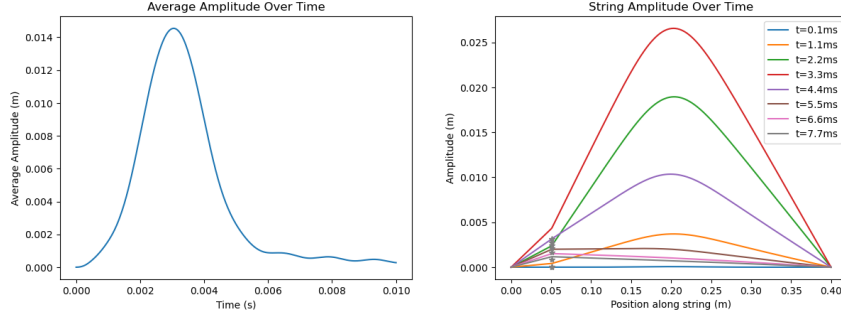


Figure 10: Reponse With Biased Damper

Wider and heavier damper empowers more effective damping. When $d = 2$ cm the response of string are significantly damped, and process of releasing energy is much slower.

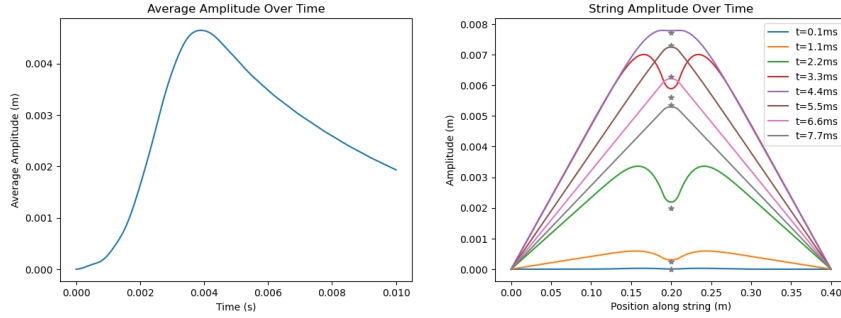


Figure 11: Reponse With wide Damper

4.2 Sensitivity Analysis

The stability analysis has already partly analyzed the sensitivity to the permutation. In this section, physical parameters below are considered:

- Impact position x_0

- Elasticity of the tennis, which influences the contact time t_d and σ_x
- Velocity of the tennis v_d
- Mass of the damper m_d

Each parameter is varied by $\pm 10\%$ and the change of the mean amplitude is measured to quantify the sensitivity. The results are shown in following tables and figures.

Table 1: Sensitivity Analysis Results: Percentage Change in Mean Amplitude

Parameter	Variation	% Change in Mean Amplitude
x_0	$\pm 75\%$	-56.90%
x_0	$\pm 50\%$	-25.78%
x_0	$\pm 25\%$	-6.56%
x_0	$\pm 10\%$	-1.03%
t_0	-10%	$+10.45\%$
t_0	$+10\%$	-9.06%
σ_x	-10%	$+0.51\%$
σ_x	$+10\%$	-0.56%
m	-10%	-10.00%
m	$+10\%$	$+10.00\%$

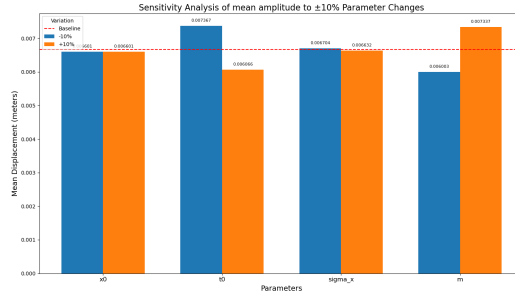


Figure 12: Sensitivity Analysis

According to the results, we can see that small change of x_0 does not affect the mean amplitude, but the influence rapidly increases with the offset of the impact position. Approximately the range of ± 5 cm from the middle point is the **sweat zone** of a string with $L = 0.4$ m.

The dynamic system is sensitive to the elasticity of the tennis. Harder the ball, stronger the shock. That's because the hard tennis has shorter contact time t_0 , thus increasing the impulse on the racket. However, because of the existence of the sweat zone, σ_x , which means the deformation of the tennis is not so critical.

The mass of the tennis is the most direct and critical property. It indicates that, although the damper reduces the shock, higher momentum can still cause stronger vibration.

5 Conclusions

In conclusion, we developed a dynamic system that models the behavior of the coupled system consists of the damper and a single string. We further analyzed relative parameters and conducted sensitivity analysis. According to the results, we can conclude that heavy and wide damper positioned in the sweat zone is capable of reducing the vibration significantly. First, the maximum amplitude of string vibrating is lower. Second, the time average vibratile energy is lower. Third, the process of releasing the energy is slowed. Fourth, the resonance is avoided. However, these conclusions are theoretical and do not take into account the technical and tactical requirements of actual athletes. More research needs to be performed to optimize the settings of the damper's parameters.

6 Acknowledgments

Usage of AI: The introduction and problem statement part is partly composed by AI (ChatGPT 4o). AI-enhanced tools are also used for searching information and papers on the Internet. Also, the codes shown below are sorted and added comments by AI. However, the main body of the paper and all codes are composed **purely by hand**.

The mathematical derivation process is partly borrowed from Qiao, Gu (2012) Mathematical Methods for Physics, which is listed in references.

7 References

- Brody, H. (1979). Physics of the tennis racket. American Journal of Physics, 47(6), 482–487.
- Gao Lu, Chen Guofang (2004). Analysis on the Relationship between String Tension and Hitting Speed of Tennis Racket. Mechanics in Engineering, 26(4)
- Qiao, Gu (2012). Mathematical Methods for Physics, Beijing, Science Press

Appendix: Critical Codes

A Analytic Solution to the Wave Equation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import cumtrapz
4
5 ## Analytic solution to  $u_{tt} = a^2 u_{xx} + f(x,t)$ 
6
7 # Physical Parameters
8 L = 0.4                # Length of the string (meters)
9 T = 311                # Tension in the string (Newtons)
10 rho = 0.00119          # Linear mass density (kg/m)
11 c = np.sqrt(T / rho)   # Wave speed (m/s)
12 m = 0.057              # Mass of tennis (kg)
13 vd = 45                # Impact velocity (m/s)
14
15 # Impact Parameters
16 x0 = 0.2                # Impact
17                          # position (meters)
18 t0 = 0.003              # Peak
19                          # impact time (seconds)
20 sigma_x = np.sqrt(2) * 0.0686 / 3  #
21                          # Spatial standard deviation (meters)
22 sigma_t = t0 / 3        #
23                          # Temporal standard deviation (seconds)
24 F0 = m * vd / (np.pi * rho * sigma_x * sigma_t * 20) #
25                          # Peak force (meter per second square)
26
27 # Further Reduced Number of Modes for Maximum Efficiency
28 N_modes = 3            # Number of modes to include in the
29                          # sum
30
31 # Further Reduced Spatial and Temporal Resolution
32 N_x = 100              # Number of spatial points
33 N_t = 100              # Number of time points
34 T_total = t0           # Total simulation time (seconds)
```

```

29
30 # Spatial and Temporal Grids
31 x = np.linspace(0, L, N_x + 1)          # Spatial grid
    points
32 t = np.linspace(0, T_total, N_t + 1) # Time grid points
33 dt = t[1] - t[0]                      # Time step size
34 dx = x[1] - x[0]                      # Spatial step
    size
35
36 def f_func(x, t_val):
37     """External force as a 2D Gaussian."""
38     return F0 * np.exp(-(((x - x0)**2) / (2 * sigma_x
        **2) + ((t_val-t0)**2) / (2 * sigma_t**2)))
39
40 n_values = np.arange(1, N_modes + 1) # Mode numbers
    from 1 to N_modes
41 omega_n = (n_values * np.pi * c) / L # Angular
    frequencies for each mode
42
43 # Initialize f_n(t) array
44 f_n_all = np.zeros((N_modes, N_t + 1))
45
46 # Compute f_n(t) for each mode and time
47 for i, n in enumerate(n_values):
48     # Using vectorized operations for efficiency
49     sine_n = np.sin(n * np.pi * x / L)
50     # Since f(x,t) is a Gaussian in both x and t, it's
        separable in space and time. The spatial integral
        can be precomputed and is independent of time
51     spatial_integral = np.trapz(np.exp(-((x - x0)**2) /
        (2 * sigma_x**2)) * sine_n, x)
52     f_n_all[i, :] = (2 * F0 / L) * spatial_integral * np
        .exp(-(t**2) / (2 * sigma_t**2))
53
54 # Initialize A_n(t) array
55 A_n_t = np.zeros((N_modes, N_t + 1)) # Shape: (N_modes,
    N_t + 1)
56
57 # Compute A_n(t) for each mode using cumulative
    trapezoidal integration

```

```

58 for i in range(N_modes):
59     # Compute the integrand:  $\sin(\omega_n * (t - \tau)) * f_n(\tau)$ 
        f_n(tau)
60
61     # Reverse the f_n(tau) array for convolution with
        sin( $\omega_n * s$ )
62     f_n_reversed = np.flip(f_n_all[i, :])
63
64     # Compute the integrand as  $\sin(\omega_n * t) * f_n(t)$ 
65     integrand = np.sin( $\omega_n[i] * t$ ) * f_n_reversed
66
67     # Perform cumulative trapezoidal integration
68     integral = cumtrapz(integrand, t, initial=0)
69
70     # Compute  $A_n(t)$ 
71     A_n_t[i, :] = integral /  $\omega_n[i]$ 
72
73 # Initialize displacement array
74 u = np.zeros((N_t + 1, N_x + 1)) # Shape: (N_t + 1, N_x
    + 1)
75
76 # Compute u(x, t) by summing over modes
77 for i in range(N_modes):
78     u += np.outer(A_n_t[i, :], np.sin(n_values[i] * np.
        pi * x / L))
79
80 # Select specific time snapshots to plot
81 time_snapshots = [0.003] # in seconds
82 indices = [np.argmin(np.abs(t - ts)) for ts in
    time_snapshots]
83
84 # Plot the Amplitude profiles at selected times
85 plt.figure(figsize=(6, 4))
86 for idx, ts in zip(indices, time_snapshots):
87     plt.plot(x, -u[idx, :], label=f't = {ts:.3f} s')
88     x_for_valid_ = x
89     u_for_valid_ = u[idx, :]
90
91 plt.xlabel('Position along string (m)')
92 plt.ylabel('Amplitude (m)')

```

```

93 plt.title('String Amplitude at Various Times')
94 plt.legend()
95 plt.grid(True)
96 plt.tight_layout()
97 plt.show()

```

Listing 1: Analytic solution to $u_{tt} = a^2 u_{xx} + f(x, t)$

B Numerical Solution to the Wave Equation

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import quad
4 from multiprocessing import Pool
5 import functools
6 import matplotlib.animation as animation
7 from scipy.integrate import cumtrapz
8 from scipy.signal import find_peaks
9 import copy
10 import pandas as pd
11
12 ## Numerical solution to  $u_{tt} = a^2 u_{xx} + f(x, t)$ 
13
14 # Parameters
15 L = 0.4 # Length of the string (meters)
16 T = 311 # Tension in the string (Newtons)
17 rho = 0.00119 # Linear mass density (kg/m)
18 c = np.sqrt(T / rho) # Wave speed (m/s)
19 m = 0.057 # Mass of tennis (kg)
20 vd = 45 # Impact velocity (m/s)
21
22 # Impact Parameters
23 x0 = 0.2 #
24     Impact position (meters)
25 t0 = 0.003 #
26     Peak impact time (seconds)
27 sigma_x = np.sqrt(2) * 0.0686 / 3 #
28     Spatial standard deviation (meters)

```

```

26 sigma_t = 0.003 / 3 #
    Temporal standard deviation (seconds)
27 F0 = m * vd / (np.pi * rho * sigma_x * sigma_t * 20) #
    Peak force (meter per second square)
28
29 # Numerical Parameters
30 N = 500 # Number of spatial divisions
31 dx = L / N # Spatial step size (meters)
32 dt = 0.9 * dx / c # Time step size (seconds) -
    Courant condition with safety factor
33 T_total = 0.016 # Total simulation time (seconds)
34 M = int(T_total / dt) # Number of time steps
35
36 lambda_sq = (T / rho) * (dt / dx)**2
37
38 # Spatial grid
39 x = np.linspace(0, L, N+1)
40
41 # Initialize displacement arrays
42 u_prev = np.zeros(N+1)
43 u_curr = np.zeros(N+1)
44 u_next = np.zeros(N+1)
45 u_free = np.zeros((M+1, N+1))
46
47 # Initial conditions
48 # u_prev = u(x,0) = 0
49 # u_curr = u(x,dt) from initial velocity = 0
50 # Compute u_curr using initial acceleration
51 t_0 = 0.0
52 for i in range(1, N):
53     f_i = F0 * np.exp(-((x[i] - x0)**2) / (2 * sigma_x
54         **2) - ((t_0-t0)**2) / (2 * sigma_t**2))
55     u_curr[i] = (u_prev[i] +
56         dt * 0 + # Initial velocity is zero
57         0.5 * (lambda_sq) * (u_prev[i+1] - 2*
58             u_prev[i] + u_prev[i-1]) +
59         0.5 * (dt**2) * f_i)
60
61 # Time-stepping loop
62 for n in range(1, M):

```



```

61     t = n * dt
62     for i in range(1, N):
63         # Compute forcing term at current time
64         f_i = F0 * np.exp(-((x[i] - x0)**2) / (2 *
65             sigma_x**2) - ((t-t0)**2) / (2 * sigma_t**2))
66         # Update displacement
67         u_next[i] = (2 * u_curr[i] - u_prev[i] +
68             lambda_sq * (u_curr[i+1] - 2*u_curr
69                 [i] + u_curr[i-1])) +
70             dt**2 * f_i)
71     u_free[n] = u_curr
72     # Apply boundary conditions
73     u_next[0] = 0
74     u_next[N] = 0
75     # Update for next step
76     u_prev, u_curr = u_curr, u_next
77     u_next = np.zeros(N+1)
78     # Visualization
79     if (n+100) % 700 == 0:
80         plt.plot(x, u_curr, label=f't={1000*t:.1f}ms')
81     if n == 2130:
82         x_for_valid = x
83         u_for_valid = u_prev
84     # Final plot
85     plt.xlabel('Position along string (m)')
86     plt.ylabel('Amplitude (m)')
87     plt.title('String Amplitude Over Time')
88     plt.legend()
89     plt.show()

```

Listing 2: Numerical solution to $u_{tt} = a^2 u_{xx} + f(x, t)$

C Numerical Solution to the Coupled Dynamic System

```

1 import numpy as np

```

```

2 import matplotlib.pyplot as plt
3 from scipy.integrate import cumtrapz
4 from numpy import seterr
5 seterr(all='raise')
6
7 # Parameters
8 L = 0.4                # Length of the string (meters)
9 T = 311                # Tension in the string (Newtons)
10 rho = 0.00119         # Linear mass density (kg/m)
11 c = np.sqrt(T / rho)  # Wave speed (m/s)
12 m = 0.057             # Mass of tennis (kg)
13 md = 0.002            # Mass of damper (kg)
14 vd = 45               # Impact velocity (m/s)
15 E = 0.078 * 10**9     # Elastic Module (Pa)
16 mu = 600              # Viscosity (Pa s)
17 xd = 0.199            # Position of damper left end (m)
18 wid = 0.002           # Width of damper (m)
19
20 # Impact Parameters
21 x0 = 0.2               #
22                         # Impact position (meters)
23 t0 = 0.003             #
24                         # Peak impact time (seconds)
25 sigma_x = np.sqrt(2) * 0.0686 / 3 #
26                         # Spatial standard deviation (meters)
27 sigma_t = 0.003 / 3    #
28                         # Temporal standard deviation (seconds)
29 F0 = m * vd / (np.pi * rho * sigma_x * sigma_t * 20) #
30                         # Peak force (meter per second square)
31
32 # Numerical Parameters
33 N = 500                # Number of spatial divisions
34 dx = L / N             # Spatial step size (meters)
35 dt = 0.07 * dx / c     # Time step size (seconds) -
36                         # Courant condition with safety factor
37 T_total = 0.006        # Total simulation time (seconds)
38 M = int(T_total / dt)  # Number of time steps
39
40 lambda_sq = (T / rho) * (dt / dx)**2

```

```

36 # Spatial grid
37 x = np.linspace(0, L, N+1)
38
39 # Initialize displacement arrays
40 u_prev = np.zeros(N+1)
41 u_curr = np.zeros(N+1)
42 u_next = np.zeros(N+1)
43 u = np.zeros((M+1, N+1))
44 y = np.zeros(M+1)
45
46 def window(x_var, left_end=xd, width=wid):
47     if x_var > left_end and x_var - left_end < width:
48         return 1
49     else:
50         return 0
51
52 # Initial conditions
53 # u_prev = u(x,0) = 0
54 # u_curr = u(x,dt) from initial velocity = 0
55 # Compute u_curr using initial acceleration
56 t_0 = 0.0
57 ytt = 0.0
58 for i in range(1, N):
59     f_i = F0 * np.exp(-((x[i] - x0)**2) / (2 * sigma_x
60         **2)) - ((t_0-t0)**2) / (2 * sigma_t**2))
61     u_curr[i] = u_prev[i] + dt * 0 + 0.5 * (lambda_sq) *
62         (u_prev[i+1] - 2*u_prev[i] + u_prev[i-1]) + 0.5
63         * (dt**2) * f_i
64     fd = window(i * dx) * (E * (y[0] - x[i]) - mu * (
65         u_curr[i] - u_prev[i]) / dt) / rho
66     ytt += fd/md
67 y[1] = ytt * dt**2
68
69 # Time-stepping loop
70 for n in range(1, M):
71     t = n * dt
72     ytt = 0.0 # acceleration of damper
73     for i in range(1, N):
74         # Compute damping term at current time

```

```

71         fd = window(i * dx) * (E * (y[n] - u_curr[i]) +
72             mu * ((y[n] - y[n-1]) / dt - (u_curr[i] -
73                 u_prev[i]) / dt)) / rho
74     # Compute forcing term at current time
75     f_i = F0 * np.exp(-((x[i] - x0)**2) / (2 *
76         sigma_x**2) - ((t-t0)**2) / (2 * sigma_t**2))
77         + fd
78     # Update displacement
79     u_next[i] = (2 * u_curr[i] - u_prev[i] +
80         lambda_sq * (u_curr[i+1] - 2 *
81             u_curr[i] + u_curr[i-1]) +
82             dt**2 * f_i)
83     # Sum damping force
84     ytt += fd * dx / md
85     u[n] = u_curr
86     # update damper position
87     y[n+1] = y[n] - ytt * dt**2
88     # Apply boundary conditions
89     u_next[0] = 0
90     u_next[N] = 0
91     # Update for next step
92     u_prev, u_curr = u_curr, u_next
93     u_next = np.zeros(N+1)
94     # Optional: Visualization
95     if (n+200) % 7000 == 0:
96         plt.plot(x, u_curr, label=f't={1000*t:.1f}ms')
97         plt.plot(xd + wid/2, y[n], '*', color='gray')
98     if n == 2130:
99         x_for_valid = x
100         u_for_valid = u_prev
101
102 # Final plot
103 plt.xlabel('Position along string (m)')
104 plt.ylabel('Amplitude (m)')
105 plt.title('String Amplitude Over Time')
106 plt.legend()
107 plt.show()
108
109 plt.figure()
110 plt.plot(np.mean(u[:-1], axis=1), y[:-1])

```

```

106 plt.xlabel('Average Amplitude')
107 plt.ylabel('Displacement')
108 plt.grid(True)
109 plt.title('Phase Trajectory')
110 plt.show()

```

Listing 3: Numerical solution to the coupled dynamic system

D Sensitivity Analysis

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import cumtrapz
4 from numpy import seterr
5 seterr(all='raise')
6 import copy
7 import pandas as pd
8
9 ## sensitivity analysis
10 def simulate_dynamics(
11     x0=0.2,
12     t0=0.003,
13     sigma_x=np.sqrt(2) * 0.0686 / 3,
14     sigma_t=0.003 / 3,
15     m=0.057,
16     verbose=False
17 ):
18     """
19     Simulates the dynamic system and returns the maximum
20         displacement mean_u.
21
22     Parameters:
23     - x0 (float): Impact position (meters)
24     - t0 (float): Peak impact time (seconds)
25     - sigma_x (float): Spatial standard deviation (
26         meters)
27     - sigma_t (float): Temporal standard deviation (
28         seconds)

```

```

26 - m (float): Mass of tennis (kg)
27 - verbose (bool): If True, prints progress messages.
28
29 Returns:
30 - max_u (float): Maximum displacement in the
    simulation.
31 """
32 # Interested Parameters
33 # x0, t0, sigma_x, sigma_t, m are passed as
    arguments
34
35 # Physical Parameters
36 L = 0.4                # Length of the string (meters
    )
37 T = 311                # Tension in the string (
    Newtons)
38 rho = 0.00119          # Linear mass density (kg/m)
39 c = np.sqrt(T / rho)   # Wave speed (m/s)
40 md = 0.002             # Mass of damper (kg)
41 vd = 45                # Impact velocity (m/s)
42 E = 0.078 * 10**9      # Elastic Module (Pa)
43 mu = 600               # Viscosity (Pa s)
44 xd = 0.199             # Position of damper left end
    (m)
45 wid = 0.002            # Width of damper (m)
46 F0 = m * vd / (np.pi * rho * sigma_x * sigma_t * 20)
    # Peak force (meter per second square)
47
48 # Numerical Parameters
49 N = 500                # Number of spatial divisions
50 dx = L / N             # Spatial step size (meters)
51 dt = 0.07 * dx / c     # Time step size (seconds) -
    Courant condition with safety factor
52 T_total = 2 * t0       # Total simulation time (
    seconds)
53 M = int(T_total / dt)  # Number of time steps
54
55 lambda_sq = (T / rho) * (dt / dx)**2
56
57 # Spatial grid

```

```

58 x = np.linspace(0, L, N+1)
59
60 # Initialize displacement arrays
61 u_prev = np.zeros(N+1)
62 u_curr = np.zeros(N+1)
63 u_next = np.zeros(N+1)
64
65 # Initialize y
66 y = 0.0 # y at current time step
67 y_prev = 0.0 # y at previous time step
68
69 def window(x_var, left_end=xd, width=wid):
70     return 1 if left_end < x_var < (left_end + width
71         ) else 0
72
73 # Initial conditions
74 t_initial = 0.0
75 # Compute u_curr using initial acceleration
76 for i in range(1, N):
77     f_i = F0 * np.exp(-((x[i] - x0)**2) / (2 *
78         sigma_x**2) - ((t_initial - t0)**2) / (2 *
79         sigma_t**2))
80     u_curr[i] = (u_prev[i] +
81         dt * 0 + # Initial velocity is
82         zero
83         0.5 * lambda_sq * (u_prev[i+1] - 2*
84         u_prev[i] + u_prev[i-1]) +
85         0.5 * (dt**2) * f_i)
86     # Damping force contribution
87     fd = window(x[i]) * (E * (y - u_prev[i]) - mu *
88         (u_curr[i] - u_prev[i]) / dt) / rho
89     ytt = fd / md # Acceleration of y
90 y = y_prev + 0.5 * ytt * dt**2 # Initial update of
91 y
92
93 # Time-stepping loop
94 for n in range(1, M):
95     t = n * dt
96     ytt = 0.0 # Total acceleration from damping
97         and elastic forces

```

```

90
91     for i in range(1, N):
92         # Compute damping term at current time
93         relative_velocity = (y - y_prev) / dt - (
94             u_curr[i] - u_prev[i]) / dt
95
96         fd = window(x[i]) * (E * (y - u_curr[i]) +
97             mu * relative_velocity) / rho
98
99         # Compute forcing term at current time
100        f_i = (F0 * np.exp(-((x[i] - x0)**2) / (2 *
101            sigma_x**2) - ((t - t0)**2) / (2 *
102            sigma_t**2))) +
103            fd)
104
105        # Update displacement
106        u_next[i] = (2 * u_curr[i] - u_prev[i] +
107            lambda_sq * (u_curr[i+1] - 2 *
108            u_curr[i] + u_curr[i-1])) +
109            dt**2 * f_i)
110
111        # Accumulate damping force for y
112        acceleration
113        ytt += fd * dx / md
114
115        # Update y using accumulated acceleration
116        y_new = 2 * y - y_prev - ytt * dt**2 # Using
117        Verlet integration
118
119        # Update maximum displacement
120        mean_u += np.mean(u_curr)
121
122        # Apply boundary conditions
123        u_next[0] = 0
124        u_next[N] = 0
125
126        # Prepare for next time step
127        u_prev, u_curr = u_curr, u_next
128        u_next = np.zeros(N+1)
129        y_prev, y = y, y_new

```



```

123     # Optional: Visualization
124     if (n+200) % 7000 == 0:
125         plt.plot(x, u_curr, label=f't={1000*t:.1f}ms
126             ')
127         plt.plot(xd + wid/2, y, '*', color='gray')
128     if n == 2130:
129         x_for_valid = x
130         u_for_valid = u_prev
131
132     # Final plot
133     plt.xlabel('Position along string (m)')
134     plt.ylabel('Amplitude (m)')
135     plt.title('String Amplitude Over Time')
136     plt.legend()
137     plt.show()
138
139     plt.figure()
140     plt.plot(np.mean(u[:-1], axis=1), y[:-1])
141     plt.xlabel('Average Amplitude')
142     plt.ylabel('Displacement')
143     plt.grid(True)
144     plt.title('Phase Trajectory')
145     plt.show()
146
147     # Baseline parameters
148     baseline_params = {
149         'x0': 0.2, # Impact
150         position (meters)
151         't0': 0.003, # Peak impact
152         time (seconds)
153         'sigma_x': np.sqrt(2) * 0.0686 / 3, # Spatial
154         standard deviation (meters)
155         'sigma_t': 0.003 / 3, # Temporal
156         standard deviation (seconds)
157         'm': 0.057 # Mass of tennis
158         (kg)
159     }
160
161     # Define the parameters to vary
162     parameters = ['x0', 't0', 'sigma_x', 'sigma_t', 'm']

```

```

157
158 # Percentage change
159 delta = 0.05 # 5%
160
161 # Create a list to hold all simulation cases
162 simulation_cases = []
163
164 for param in parameters:
165     for change in [-delta, delta]: # -5% and +5%
166         # Create a copy of baseline parameters
167         params = copy.deepcopy(baseline_params)
168         # Apply the percentage change
169         params[param] *= (1 + change)
170         # Label the case
171         label = f"{param} {'-5%' if change < 0 else
                    '+5%'}"
172         simulation_cases.append({'label': label, 'params':
                                params})
173
174 # Add baseline case
175 simulation_cases.append({'label': 'Baseline', 'params':
                        baseline_params})
176
177 # Dictionary to store results
178 results = {}
179
180 for case in simulation_cases:
181     label = case['label']
182     params = case['params']
183     print(f"Running simulation: {label}")
184     max_u = simulate_dynamics(
185         x0=params['x0'],
186         t0=params['t0'],
187         sigma_x=params['sigma_x'],
188         sigma_t=params['sigma_t'],
189         m=params['m'],
190         verbose=True
191     )
192     results[label] = max_u

```

```

193     print(f"Mean Amplitude for {label}: {max_u:.6f}
194           meters\n")
195
196 # Create a DataFrame from the results
197 df_results = pd.DataFrame(list(results.items()), columns
198                             =['Case', 'Max_u (meters)'])
199 df_results = df_results.set_index('Case').sort_index()
200
201 print(df_results)
202
203 # Organize the results for plotting
204 parameters = ['x0', 't0', 'sigma_x', 'sigma_t', 'm']
205 variations = ['-5%', '+5%']
206
207 param_list = []
208 variation_list = []
209 max_u_list = []
210
211 for param in parameters:
212     for var in variations:
213         key = f"{param} {var}"
214         param_list.append(param)
215         variation_list.append(var)
216         max_u_list.append(results.get(key, None)) # Use
217         None if key not found
218
219 # Create a DataFrame
220 df_plot = pd.DataFrame({
221     'Parameter': param_list,
222     'Variation': variation_list,
223     'Max_u (meters)': max_u_list
224 })
225
226 # Ensure there are no missing values
227 df_plot = df_plot.dropna()
228
229 # Define parameters and variations for plotting
230 parameters = df_plot['Parameter'].unique()
231 n_params = len(parameters)
232 n_vars = len(variations)

```

```

230
231 # Set the width of each bar
232 bar_width = 0.35
233
234 # Set positions of bars on the x-axis
235 indices = np.arange(n_params)
236
237 # Create a figure and axis
238 fig, ax = plt.subplots(figsize=(14, 8))
239
240 # Define colors for variations
241 colors = ['#1f77b4', '#ff7f0e'] # Blue for -5%, Orange
    for +5%
242
243 # Plot each variation
244 for i, var in enumerate(variations):
245     # Extract data for the current variation
246     data = df_plot[df_plot['Variation'] == var]['Max_u (
        meters)'].values
247     # Calculate the position offset
248     offset = (i - n_vars / 2) * bar_width + bar_width /
        2
249     # Plot the bars
250     ax.bar(indices + offset, data, bar_width, label=var,
        color=colors[i])
251
252 # Add horizontal line for Baseline
253 ax.axhline(y=results['Baseline'], color='r', linestyle='
    --', label='Baseline')
254
255 # Set labels and title
256 ax.set_xlabel('Parameters', fontsize=14)
257 ax.set_ylabel('Maximum Displacement (meters)', fontsize
    =14)
258 ax.set_title('Sensitivity Analysis of Mean Amplitude to
    \pm5% Parameter Changes', fontsize=16)
259
260 # Set x-ticks and labels
261 ax.set_xticks(indices)
262 ax.set_xticklabels(parameters, fontsize=12)

```

```

263
264 # Add legend
265 ax.legend(title='Variation')
266
267 # Annotate bars with their values
268 for i in range(n_params):
269     for j, var in enumerate(variations):
270         # Find the corresponding value
271         value = df_plot[(df_plot['Parameter'] ==
272                             parameters[i]) & (df_plot['Variation'] == var
273 )]['Max_u (meters)'].values
274         if len(value) > 0:
275             value = value[0]
276             # Calculate the bar position
277             bar_x = indices[i] + (j - n_vars / 2) *
278                 bar_width + bar_width / 2
279             bar_y = value
280             ax.text(bar_x, bar_y + 0.0001, f"{value:.6f}
281                 ", ha='center', va='bottom', fontsize=8)
282
283 # Adjust layout for better fit
284 plt.tight_layout()
285
286 # Show the plot
287 plt.show()

```

Listing 4: Sensitivity analysis of the dynamic system